

Secure Multi-Party Computation with Trusted Execution Environments

Sebastian David Albu

June 25, 2026

Motivation and Background

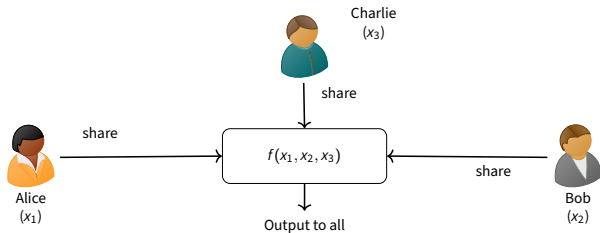
What is Secure Multi-Party Computation (MPC)?

Concept

Multiple parties jointly compute a function $f(x_1, \dots, x_n)$ without revealing their private inputs [EKR18].

Key Properties:

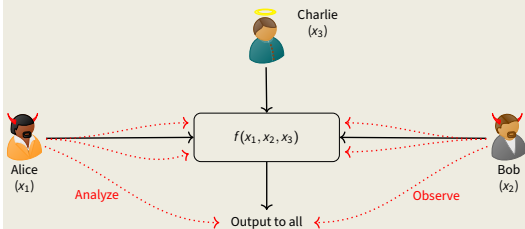
- **Privacy:** Inputs remain hidden.
- **Correctness:** Output is accurate.



Threat Models in Secure MPC

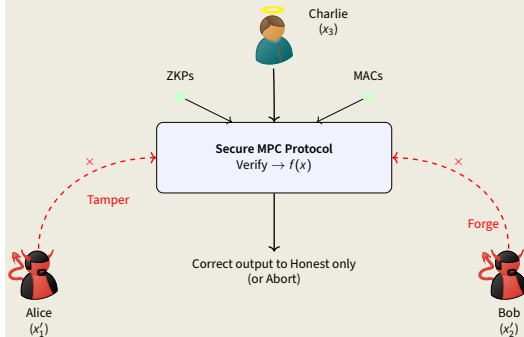
1. Semi-Honest (Passive)

"Honest-but-curious": Parties follow the protocol exactly, but passively analyze messages to infer private inputs.



2. Malicious (Active)

Adversaries can deviate **arbitrarily** (tamper, forge). Defense requires integrated checks (ZKPs, MACs).



Semi-Honest (Passive) Security

- ✓ **Very Fast:** High throughput.
- ✓ **Simple:** Standard protocols.
- ✗ **Weak Trust Model:** Assumes adversaries will not deviate from the protocol [EKR18; Bea97].

Malicious (Active) Security

- ✓ **Strong Trust Model:** Secure even if adversaries actively manipulate data.
- ✗ **The Cryptographic Cost:** Requires heavy ZKPs (computation) or MACs (network rounds).
- ✗ **Result:** Often 10 – 100× slower [Dam+12b; Dam+12a; Hwa+25].

Goal: Achieve Malicious Security guarantees with Semi-Honest performance.

If we can trust the **execution platform**, we don't need expensive cryptographic checks (ZKPs).

■ **Trusted Execution Environments (TEEs):**

- Intel SGX [Int23], Intel TDX [Int25], AMD SEV-SNP [AMD24].
- Provide **Isolation** (Confidentiality) and **Attestation** (Integrity) [Mis+24].

The "Aramis" [Kum+19] Concept

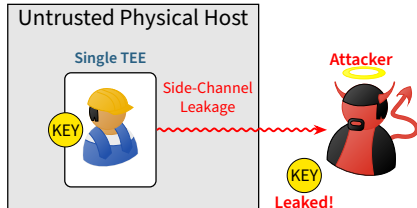
Run a Semi-Honest protocol inside a TEE. The TEE guarantees the code hasn't been tampered with [Kum+19; Dis23].

Semi-Honest Protocol + TEE Integrity $\approx$ Malicious Security

The Intuition: Why Combine MPC and TEEs?

The Problem: Single Point of Failure

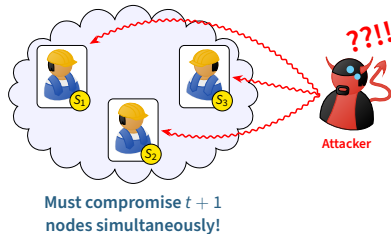
If we rely on a **single TEE**, one successful side-channel attack compromises the entire master key.



The Solution: Distributed Trust

By running **MPC over multiple TEEs**, an attacker must compromise $t + 1$ distinct hardware units simultaneously [BS26].

Untrusted MPC Cluster

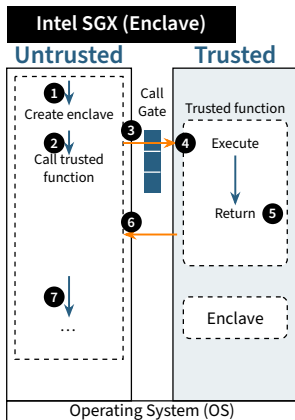


Trusted Execution Environments (TEEs)

The Shift: From Enclaves to Confidential VMs (CVMs)

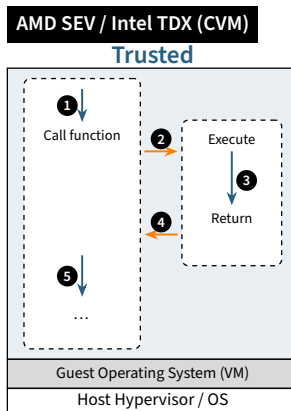
Gen 1: Enclaves (Intel SGX)

Protects specific functions. Requires heavy code rewrites to bridge via "Call Gates" [Göt+18].



Gen 2: CVMs (AMD SEV / Intel TDX)

Protects the **entire Virtual Machine**. Enables seamless "Lift & Shift" deployment [Göt+18].

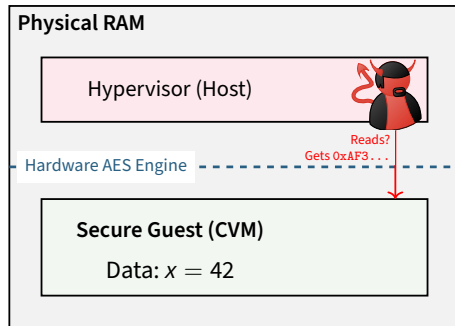


Confidential Computing

Hardware-enforced isolation of the VM from the untrusted Hypervisor [AMD24; Int25].

Key Mechanisms:

- 🔒 **Encryption:** Hardware AES engine encrypts RAM on the fly.
- 🚫 **No Injection:** Host cannot alter guest CPU registers or state.
- 🛡️ **Integrity:** Strict paging controls prevent memory rollback & replay.



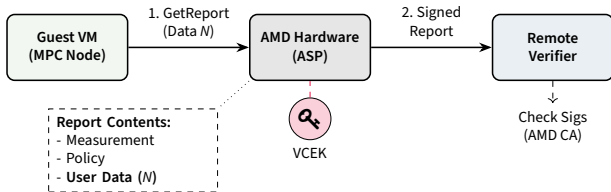
The Trust Anchor: Remote Attestation

Trusting the Hardware:

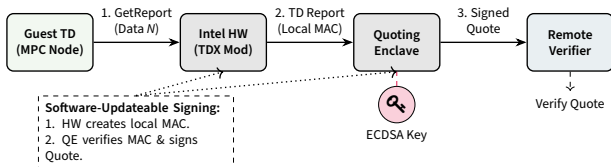
To replace ZKPs, the remote parties must verify that the correct MPC software is running inside a genuine CVM.

- 1 **Measure:** Hardware (ASP or TDX Module) measures the boot state/binary.
- 2 **Bind:** Guest binds custom data (e.g., MPC public keys) to the report.
- 3 **Sign:** Hardware/Enclave signs the report using a vendor-certified key.

AMD SEV-SNP (Hardware-Signed) [Kol22; Adv25]



Intel TDX (Two-Stage / QE-Signed) [Kol22; Int25]



Threat Model & Deployment Landscapes

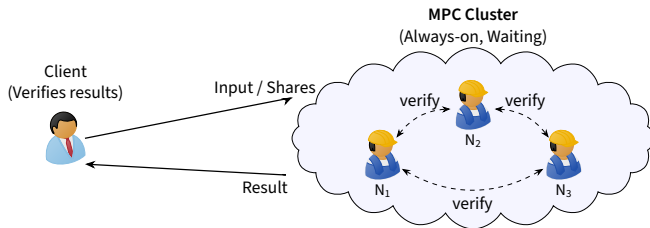
System Model: Delegated MPC [Urb24]

Outsourced Computation

Clients (lightweight) offload heavy processing to an MPC Cluster [Urb24].

Always-On Service

- Servers are **already running**.
- Waiting for Client requests.
- **Benefit:** No setup latency.



To securely delegate MPC to the cloud, we must explicitly define our trust boundaries.

Untrusted (In-Scope Threats)

-  **Malicious Peers:** Participating MPC nodes may collude, tamper with data, or forge messages.
-  **The Cloud Provider (CSP):** The host Hypervisor/OS is assumed to be fully malicious (reads RAM, manipulates network traffic).
-  **The Network:** All WAN communication is subject to interception and replay attacks.

Trusted (Our Anchors)

-  **Vendor Silicon:** We assume Intel and AMD CPU hardware correctly enforces memory encryption [AMD24; Int25].
-  **Root of Trust (RoT):** We trust the cryptographic PKI chains provided by the hardware vendors.

Defining the Trusted Computing Base (TCB)

The **TCB** represents everything that **must** operate flawlessly for the system to remain secure.

The Paradigm Shift

Gen 1: Enclaves (Intel SGX)

- **TCB:** CPU Silicon + Specific Enclave App.
- **Result:** Tiny attack surface, but requires heavy code rewrites [Kol22; Göt+18].

Gen 2: CVMs (SEV-SNP / TDX)

- **TCB:** CPU Silicon + **Full Guest OS** + App.
- **Result:** "Lift and shift" compatibility, but we now inherently trust the Guest OS [Kol22; Göt+18].

TCB Across Deployment Models

Component	Bare-Metal	IaaS (CVM)	PaaS
Hardware Silicon	✓	✓	✓
Host Hypervisor	✗	✗	✗
Guest OS Kernel	✓ (Self)	✓ (Self)	✓ (CSP)
Orchestration	✓ (Self)	✓ (Self)	✓ (CSP)
MPC Application	✓	✓	✓

i Note on PaaS: In services like Confidential Space, we partially trust the Cloud Provider (CSP) to engineer the hardened Guest OS.

Deployment Architecture and Implementation

Managed TEEs (PaaS)

Services like **Google Confidential Space** [Goo26] or **AWS Nitro Enclaves** [Ama26] abstract hardware complexity via high-level attestation APIs.

The Advantages:

- Lower operational TCB (Attestation is handled by the CSP) [Lut+25; Ama26; Goo26].

Architectural Constraints

Managed environments often enforce **egress-only** or proxied networking to minimize the data exfiltration surface.

Impact on Interactive MPC:

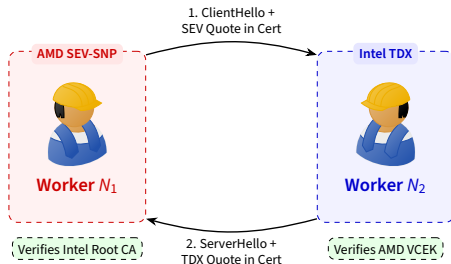
- **Bidirectional P2P:** High-round protocols (MP-SPDZ) require nodes to act as both clients and servers simultaneously.

Observation: IaaS/Bare-Metal currently provide the most flexible P2P environment.

Bootstrapping the Mesh: Mutual Attestation

To execute MPC, nodes must trust **each other**, not just a client. We designed a custom mesh initialization phase:

- ↔ **Mutual TLS (mTLS):** Standard encrypted socket connections between all peers.
- ⚙ **Hardware Binding:** Nodes embed their hardware quotes (TDX/SEV) directly into the TLS X.509 certificate extensions.
- 🔔 **Exact-Match CRL:** Nodes strictly verify vendor Certificate Revocation Lists before accepting a socket.
- 🕒 **Freshness (Nonces):** Exchanges include randomized nonces bound to the quotes to prevent attestation replay attacks.



The Attestation Toolchain: Bridging the Vendor Gap

To orchestrate heterogeneity, we engineered a unified layer over completely different vendor architectures, combining custom binaries with standard open-source tools.

AMD SEV-SNP Toolchain

Quote Generation:

- **Tool:** Custom Go binary utilizing Google's `go-sev-guest` API [Goo24a].
- **Method:** Interfaces directly with the `/dev/sev-guest` kernel device to mint quotes.

Verification & Collateral:

- **Tools:** VirTEE's `snpquest` CLI [Com23] + `OpenSSL`.
- **Method:** `snpquest` dynamically queries the AMD KDS for the VCEK and checks signatures. `OpenSSL` explicitly parses the Exact-Match CRL.

Intel TDX Toolchain

Quote Generation:

- **Tool:** Custom `tdx_tool` binary.
- **Method:** Writing the hash commitment directly into the Linux TSM ConfigFS (`inblob/outblob`).

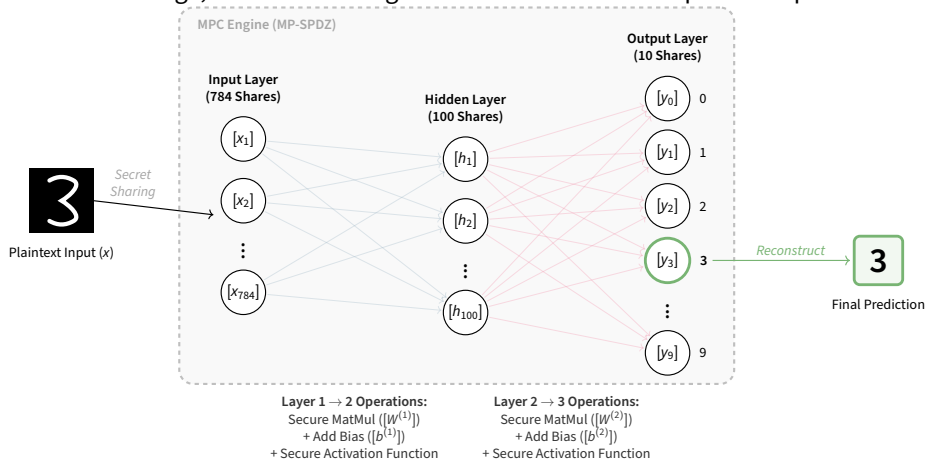
Verification & Collateral:

- **The Gap:** No standard CLI exists for DCAP Quote V4 verification.
- **Our Fix:** Cross-compiled a standalone verifier using Google's `go-tdx-guest` [Goo24b].

Use Cases

Evaluation Use Case: Privacy-Preserving ML

The Scenario: We evaluate training a Neural Network on the MNIST [Den12] dataset. The client secret-shares the image, and the MPC engine trains the NN and computes the prediction blindly.



Context: Zero-Knowledge Circuits

To evaluate our CVM architecture, we benchmarked a heavy cryptographic workload: generating Zero-Knowledge Proofs (ZKPs).

The Languages

- **Circom:** A widely used, low-level language that compiles to R1CS.
- **Noir:** A modern, Rust-like language for zero-knowledge programs.

The Standard Limitation

Standard Circom and Noir provers assume a **single entity** holds all the private data (the "Witness") to generate the ZKP.

The Problem: What if the private data required for the ZKP is split across 3 competing banks or hospitals?

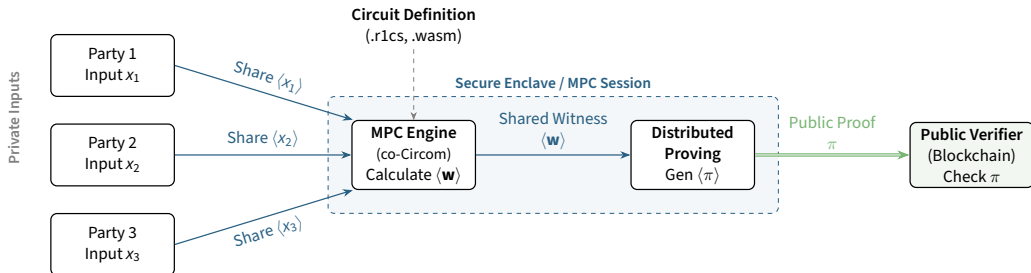
↓ This leads us to Collaborative Proving (*co-circom* / *co-noir*).

Collaborative ZKPs: co-circom & co-noir

The Concept: Developed by TACEO, these frameworks enable multiple mutually distrusting parties to jointly generate a Zero-Knowledge Proof (ZKP) over their combined private data [TAC24b; TAC24a].

- ⚙️ co-circom: Evaluates Circom circuits.
- ⚙️ co-noir: Evaluates Noir circuits.

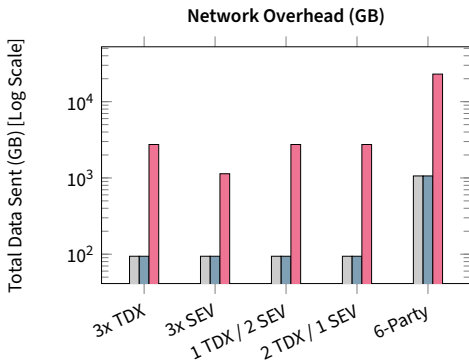
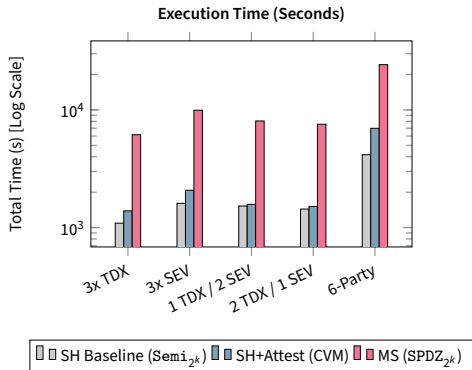
🛡️ **The Benefit:** The MPC engine evaluates the circuit distributively. No single party ever sees the full, combined plaintext data (the Witness).



Evaluation Results

Results: High-Level PPML (MNIST Training)

Goal: Train a Dense Neural Network on MNIST (Only 16 images). This highlights the communication bottleneck of traditional Malicious Security (MS). The TEE Mesh reduces execution time by up to 79% and network bandwidth by 96% compared to traditional cryptography.



The Attestation Threshold: Algorithmic Scaling

Task: Low-level C++ array manipulations (`simple.cpp`).

Defining the Threshold

At lower dataset sizes ($N = 100$), the fixed hardware attestation penalty makes the CVM Mesh slower than pure Malicious Security.

However, as the dataset scales ($N = 10,000$), pure cryptography scales exponentially. The CVM Mesh scales linearly with Semi-Honest arithmetic, easily overtaking the MS baseline.

Table: `simple.cpp` on 3x Intel TDX Mesh

Security Model	Small ($N = 100$)		Large ($N = 10,000$)	
	Attest (s)	Total (s)	Attest (s)	Total (s)
MS (SPDZ _{2^k})	N/A	132.78	N/A	711.86
SH+Attest (CVM)	56.56	185.75	32.18	338.05

TEEs require a minimum computational depth to amortize the fixed network attestation penalty.

co-SNARKs Results: Memory Encryption Equivalence

Table: co-Noir (UltraHonk) on 3x Intel TDX Cloud Mesh

Circuit	Security Model	Setup (s)	Proof Math (ms)	Net (MB)
poseidon	Plain (No Security)	N/A	490.00 ms	276 MB
	SH+Attest (CVM)	201.02 s	489.45 ms	287 MB
aes128	Plain (No Security)	N/A	1,079.90 ms	229 MB
	SH+Attest (CVM)	282.66 s	1,077.58 ms	299 MB

Finding

Hardware memory encryption introduces **near-zero computational overhead** to heavy polynomial mathematics. Proof generation in a CVM operates at bare-metal speeds. The only penalty is the upfront orchestration handshake ($\approx 250s$) and a flat ≈ 15 MB bandwidth premium for transmitting certificates.

Conclusion & Future Work

-  **Hybrid Architecture:** Fusing Semi-Honest MPC with Confidential VMs provides Malicious Security guarantees at scalable speeds.
-  **Heterogeneous Mesh:** Bridging Intel TDX and AMD SEV-SNP under a unified mTLS attestation envelope mitigates single-platform hardware zero-days.
-  **Significant ML Acceleration:** Achieved up to 79% faster execution and 96% less bandwidth for deep learning topologies compared to traditional SPDZ_{2^k}.
-  **The Attestation Threshold:** Fixed hardware orchestration taxes render TEEs counterproductive for highly optimized protocols or short transient circuits.

System Limitations

- 🔌 **Residual CSP Trust (Availability):** While the CSP cannot read the RAM, they control the physical network and hypervisor. The system is inherently vulnerable to Denial-of-Service (DoS) and liveness failures.
- 🔒 **The Hardware Assumption:** The malicious security guarantee is entirely anchored to the silicon. If a critical CVM zero-day is exploited, the security model degrades to the underlying Semi-Honest baseline.
- ⚖️ **Protocol Constraints:** co-SNARKs rely on honest-majority underlying protocols. Furthermore, our execution baseline (SPDZ_{2^k}) was a harsh dishonest-majority protocol.
- ☁️ **PaaS Scope:** Our analysis of Managed PaaS (e.g., AWS Nitro) was strictly theoretical.

Future Work

- ↔️ **PaaS Proxies:** Engineering attestation sidecars to tunnel bidirectional TCP through strictly egress-limited PaaS environments.
- ⚙️ **CVM Deployment Mesh:** Utilizing and evaluating TEEs deployed over Bare-Metal, IaaS and PaaS.
- 🔒 **Side-Channels:** Checking if a TEE mesh gives computational security against side-channel vulnerabilities.

Secure Multi-Party Computation with Trusted Execution Environments

Sebastian David Albu

June 25, 2026

Appendix A: Full High-Level Benchmarks (MNIST Dense)

Environment	Security Model	Setup/Attest	Setup Net	Compute	Compute Data	Total Time
MNIST Dense (Compute Data represented in GB)						
3x GCP (Intel TDX)	SH Baseline	N/A	N/A	1,090.91 s	94.17 GB	1,090.91 s
	SH+Attest	15.00 s	128.00 MB	1,372.83 s	94.17 GB	1,387.83 s
	MS (SPDZ _{2^k})	N/A	N/A	6,156.51 s	2,744.70 GB	6,156.51 s
3x GCP (AMD SEV)	SH Baseline	N/A	N/A	1,608.05 s	94.17 GB	1,608.05 s
	SH+Attest	30.00 s	374.90 MB	2,045.54 s	94.17 GB	2,075.54 s
	MS (SPDZ _{2^k})	N/A	N/A	9,946.75 s	1,135.27 GB	9,946.75 s
Hetero (1 TDX / 2 SEV)	SH Baseline	113.46 s	92.54 MB	1,412.04 s	94.17 GB	1,525.50 s
	SH+Attest	152.44 s	132.15 MB	1,423.30 s	94.17 GB	1,575.74 s
	MS (SPDZ _{2^k})	N/A	N/A	8,056.28 s	2,744.70 GB	8,056.28 s
Hetero (2 TDX / 1 SEV)	SH Baseline	108.30 s	92.44 MB	1,329.61 s	94.17 GB	1,437.91 s
	SH+Attest	194.14 s	133.13 MB	1,317.71 s	94.17 GB	1,511.85 s
	MS (SPDZ _{2^k})	N/A	N/A	7,560.28 s	2,744.70 GB	7,560.28 s
Hetero 6-Party (3x3)	SH Baseline	166.90 s	807.97 MB	3,995.56 s	1,062.69 GB	4,162.46 s
	SH+Attest	292.42 s	1,617.75 MB	6,691.12 s	1,062.69 GB	6,983.54 s
	MS (SPDZ _{2^k})	N/A	N/A	24,258.90 s	22,990.70 GB	24,258.90 s

Appendix A: Full High-Level Benchmarks (AES-128)

Environment	Security Model	Setup/Attest	Setup Net	Compute	Compute Data	Total Time
AES-128 (All Data represented in MB)						
3x GCP (Intel TDX)	SH Baseline	95.30 s	91.74 MB	0.07 s	22.60 MB	95.37 s
	SH+Attest	137.80 s	109.92 MB	0.08 s	22.60 MB	137.87 s
	MS (SPDZ _{2^k})	117.50 s	147.31 MB	0.26 s	141.23 MB	117.76 s
3x GCP (AMD SEV)	SH Baseline	95.37 s	91.75 MB	0.12 s	22.60 MB	95.49 s
	SH+Attest	134.11 s	116.64 MB	0.12 s	22.60 MB	134.22 s
	MS (SPDZ _{2^k})	120.75 s	147.65 MB	0.38 s	141.23 MB	121.12 s
Hetero (1 TDX / 2 SEV)	SH Baseline	109.46 s	99.15 MB	0.10 s	22.60 MB	109.56 s
	SH+Attest	159.98 s	143.08 MB	0.10 s	22.60 MB	160.08 s
	MS (SPDZ _{2^k})	134.42 s	153.93 MB	0.32 s	141.23 MB	134.74 s
Hetero (2 TDX / 1 SEV)	SH Baseline	111.40 s	92.93 MB	0.09 s	22.60 MB	111.49 s
	SH+Attest	164.25 s	147.31 MB	0.09 s	22.60 MB	164.34 s
	MS (SPDZ _{2^k})	153.18 s	158.09 MB	0.31 s	141.23 MB	153.49 s
Hetero 6-Party (3x3)	SH Baseline	263.80 s	193.38 MB	0.14 s	112.53 MB	263.94 s
	SH+Attest	341.02 s	290.19 MB	0.19 s	112.53 MB	341.21 s
	MS (SPDZ _{2^k})	620.13 s	327.35 MB	0.59 s	705.50 MB	620.72 s

Appendix B: Full Low-Level Benchmarks (Homogeneous)

Environment	Array (N)	Security Model	Attest (s)	Compute (s)	Data (MB)	Total Time (s)
3x GCP (Intel TDX)	100	SH (Plain Baseline)	N/A	129.19	171.63	129.19
		SH+Attest (TDX)	56.56	129.19	212.31	185.75
		MS (SPDZ _{2^k})	N/A	132.78	169.70	132.78
	1,000	SH (Plain Baseline)	N/A	131.80	178.20	131.80
		SH+Attest (TDX)	46.97	131.80	215.63	178.77
		MS (SPDZ _{2^k})	N/A	186.31	173.39	186.31
	10,000	SH (Plain Baseline)	N/A	305.87	166.93	305.87
		SH+Attest (TDX)	32.18	305.87	191.92	338.05
		MS (SPDZ _{2^k})	N/A	711.86	257.37	711.86
3x GCP (AMD SEV)	100	SH (Plain Baseline)	N/A	143.91	161.10	143.91
		SH+Attest (SEV)	13.44	143.91	194.65	157.35
		MS (SPDZ _{2^k})	N/A	168.43	171.71	168.43
	1,000	SH (Plain Baseline)	N/A	144.79	161.34	144.79
		SH+Attest (SEV)	74.52	144.79	184.64	219.31
		MS (SPDZ _{2^k})	N/A	248.87	162.29	248.87
	10,000	SH (Plain Baseline)	N/A	424.28	162.62	424.28
		SH+Attest (SEV)	2.78	424.28	185.44	427.06
		MS (SPDZ _{2^k})	N/A	959.06	308.66	959.06

Appendix B: Full Low-Level Benchmarks (Heterogeneous) isec.tugraz.at ■

Environment	Array (N)	Security Model	Attest (s)	Compute (s)	Data (MB)	Total Time (s)
Hetero (2 TDX / 1 SEV)	100	SH (Plain Baseline)	N/A	133.92	161.43	133.92
		SH+Attest (TEE Mesh)	120.85	133.92	203.70	254.77
		MS (SPDZ _{2^k})	N/A	144.40	161.88	144.40
	1,000	SH (Plain Baseline)	N/A	172.62	162.24	172.62
		SH+Attest (TEE Mesh)	48.74	172.62	202.62	221.36
		MS (SPDZ _{2^k})	N/A	203.65	161.80	203.65
	10,000	SH (Plain Baseline)	N/A	398.87	163.56	398.87
		SH+Attest (TEE Mesh)	59.65	398.87	203.81	458.52
		MS (SPDZ _{2^k})	N/A	818.92	164.41	818.92
Hetero (1 TDX / 2 SEV)	100	SH (Plain Baseline)	N/A	141.87	182.09	141.87
		SH+Attest (TEE Mesh)	48.78	141.87	222.23	190.65
		MS (SPDZ _{2^k})	N/A	138.30	175.55	138.30
	1,000	SH (Plain Baseline)	N/A	172.38	161.68	172.38
		SH+Attest (TEE Mesh)	40.83	172.38	229.57	213.21
		MS (SPDZ _{2^k})	N/A	208.69	194.91	208.69
	10,000	SH (Plain Baseline)	N/A	468.49	258.13	468.49
		SH+Attest (TEE Mesh)	45.60	617.28	164.41	662.88
		MS (SPDZ _{2^k})	N/A	891.25	165.01	891.25

Appendix C: co-Noir Benchmarks

Environment	Circuit	Security Model	Setup (s)	Witness (ms)	Proof (ms)	Total Net (MB)
co-Noir Framework (UltraHonk)						
3x GCP (Intel TDX)	aes128	Plain Baseline	205.69	44.27	1,079.90	229.41
		SH+Attest (TDX)	282.66	43.21	1,077.58	299.43
	bb_sha256	Plain Baseline	316.51	86.59	5,415.11	227.35
		SH+Attest (TDX)	327.86	86.77	5,412.92	270.49
3x GCP (AMD SEV)	aes128	Plain Baseline	299.53	56.01	1,261.61	246.37
		SH+Attest (SEV)	381.17	56.23	1,263.51	254.60
	bb_sha256	Plain Baseline	352.74	114.47	6,948.31	227.47
		SH+Attest (SEV)	486.50	113.17	6,807.45	253.34
Hetero Mesh (2 TDX / 1 SEV)	aes128	Plain Baseline	268.29	45.62	1,127.29	228.57
		SH+Attest (Mesh)	522.51	46.18	1,130.35	275.37
	bb_sha256	Plain Baseline	361.28	90.00	5,843.04	228.43
		SH+Attest (Mesh)	668.22	89.53	5,923.28	265.04
Hetero Mesh (1 TDX / 2 SEV)	aes128	Plain Baseline	250.83	53.16	1,159.62	228.32
		SH+Attest (Mesh)	490.03	52.89	1,163.89	276.27
	bb_sha256	Plain Baseline	399.56	99.24	6,049.91	229.14
		SH+Attest (Mesh)	742.73	100.67	6,071.29	276.14

Appendix D: The Cost of Malicious Cryptography

Protocol Overhead Breakdown

Empirical performance comparison of Plain Semi-Honest, the Hybrid CVM Mesh (**SH+Attest**), and pure Malicious Secure (SPDZ_{2^k}) MPC for the MNIST Dense workload on Bare-Metal.

Security Model	Deployment Model	Setup (s)	Compute (s)	Compute Data	Total Time (s)
SH	Plain Local	N/A	1,007.83	353.97 GB	1,007.95
SH+Attest	Local CVMs	184.59	2,148.85	353.97 GB	2,360.90
MS (SPDZ_{2^k})	Plain Local	N/A	9,628.70	3,399.68 GB	9,629.10
Malicious Overhead (MS vs SH+Attest)				$\approx 9.6\times$	$\approx 4\times$

Public Cloud (GCP) Instances

Instances were identically matched in vCPU and RAM to ensure hardware parity during heterogeneous benchmarks.

- **AMD SEV-SNP:** Google Cloud N2D instances running AMD EPYC Milan processors.
- **Intel TDX:** Google Cloud C3 instances running Intel Sapphire Rapids processors.

Bare-Metal Server

Used exclusively for loopback benchmark to isolate raw CVM virtualization overhead.

- **CPU:** AMD EPYC 9015 (Genoa architecture).
- **Network:** 127.0.0.1 loopback to eliminate WAN latency.
- **OS:** Linux kernel with KVM/QEMU SEV-SNP support.

The QEMU NAT Routing Problem

Problem: On the bare-metal server, all CVMs run behind a local QEMU NAT. MP-SPDZ natively attempts to bind and connect via local loopback (127.0.0.1), which fails to cross the hypervisor port-forwarding boundaries.

The Constraint: We needed to rewrite the destination IPs at runtime **without** altering or recompiling the massive MP-SPDZ C++ codebase.

Our Solution: User-Space Interception

Implementation: We wrote a custom C-library (`shim_redirect.so`) injected into the CVM via Linux LD_PRELOAD.

How it works:

- Intercepts the standard `libc connect()` system call.
- If the target is 127.0.0.1, it dynamically rewrites the destination to the QEMU gateway IP (10.0.2.2).
- Passes execution back to the real `libc` function, seamlessly routing the traffic out of the NAT.

The Storage Vulnerability

CVMs encrypt memory, but any data written to the cloud's persistent disk is visible to the CSP. Furthermore, standard data injection over SSH is vulnerable to MitM by the hypervisor.

Our Solution: Volatile E2EE

-  **RAM Disk (`tmpfs`):** All execution is forced into volatile memory. OS swapping is explicitly disabled.
-  **Hardware Binding:** The node generates an age decryption key **inside** the RAM disk and binds its hash to the hardware quote.
-  **Result:** The client encrypts the payload **before** transmission.

The TOCTOU Vulnerability

On bare-metal, the host mounts the MPC binary into the CVM. A malicious host could swap the binary **after** it is measured but **before** it executes (Time-of-Check to Time-of-Use).

Our Solution: In-Memory Locking

- 1 Create deterministic, reproducible `tar.gz` payloads.
- 2 CVM pulls payload directly into isolated RAM using `memfd_create`.
- 3 Execute directly from memory. **Host disk is bypassed entirely.**

- [Adv25] Advanced Micro Devices. **SEV Secure Nested Paging Firmware ABI Specification.** Specification 56860. May 2025. URL: <https://www.amd.com/content/dam/amd/en/documents/developer/56860.pdf> (visited on 05/07/2026).
- [Ama26] Amazon Web Services. **AWS Nitro Enclaves.** Accessed: 2026-04-08. 2026. URL: <https://aws.amazon.com/ec2/nitro/nitro-enclaves/>.
- [AMD24] AMD Inc. **AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More.** White paper. High-level description of SEV-ES and SEV-SNP features. 2024.
- [Bea97] Donald Beaver. **Security of Multi-Party Protocols.** CRYPTO. 1997.
- [BS26] Katharina Boudgoust and Mark Simkin. **Scale, Round, Break: Simple Leakage Attacks on Secret Sharing Schemes.** [Cryptology ePrint Archive](#) (2026).
- [Com23] VirTEE Community. **snpguest: A CLI tool for interacting with SEV-SNP guest environment.** Accessed: 2025-09-11. 2023. URL: <https://github.com/virtee/snpguest> (visited on 09/11/2025).

- [Dam+12a] Ivan Damgård et al. **Multiparty Computation from Somewhat Homomorphic Encryption**. Lecture Notes in Computer Science 7417 (2012). Ed. by Reihaneh Safavi-Naini and Ran Canetti, pp. 643–662. DOI: [10.1007/978-3-642-32009-5_38](https://doi.org/10.1007/978-3-642-32009-5_38). URL: https://doi.org/10.1007/978-3-642-32009-5_38.
- [Dam+12b] Ivan Damgård et al. **Practical Covertly Secure MPC for Dishonest Majority - or: Breaking the SPDZ Limits**. Tech. rep. 2012, p. 642. URL: <http://eprint.iacr.org/2012/642>.
- [Den12] Li Deng. **The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]**. **IEEE Signal Process. Mag.** 29.6 (2012), pp. 141–142. DOI: [10.1109/MSP.2012.2211477](https://doi.org/10.1109/MSP.2012.2211477). URL: <https://doi.org/10.1109/MSP.2012.2211477>.
- [Dis23] Juri Dispan. **Confidential Computing via Multiparty Computation and Trusted Computing**. (2023). Master's Thesis at Universität Ulm Germany.

- [EKR18] David Evans, Vladimir Kolesnikov, and Mike Rosulek. **A Pragmatic Introduction to Secure Multi-Party Computation.** *Found. Trends Priv. Secur.* 2.2-3 (2018), pp. 70–246. DOI: [10.1561/33000000019](https://doi.org/10.1561/33000000019). URL: <https://doi.org/10.1561/33000000019>.
- [Goo24a] Google. **go-sev-guest: A Go library for AMD SEV-SNP guest attestation.** <https://github.com/google/go-sev-guest>. Accessed: 2024-04-25. 2024.
- [Goo24b] Google. **go-tdx-guest: Go library for Intel TDX Guest APIs.** <https://github.com/google/go-tdx-guest>. Accessed: 2026-04-10. 2024.
- [Goo26] Google Cloud. **Confidential Space: Secure Collaboration.** Accessed: 2026-04-08. 2026. URL: <https://cloud.google.com/confidential-computing/confidential-space>.

- [Göt+18] Christian Göttel et al. **Security, Performance and Energy Trade-Offs of Hardware-Assisted Memory Protection Mechanisms.** 37th IEEE Symposium on Reliable Distributed Systems, SRDS 2018, Salvador, Brazil, October 2-5, 2018. IEEE Computer Society, 2018, pp. 133–142. DOI: [10.1109/SRDS.2018.00024](https://doi.org/10.1109/SRDS.2018.00024). URL: <https://doi.org/10.1109/SRDS.2018.00024>.
- [Hwa+25] Intak Hwang et al. **Provably Secure Approximate Computation Protocols from CKKS.** *IACR Cryptol. ePrint Arch.* (2025), p. 395. URL: <https://eprint.iacr.org/2025/395>.
- [Int23] Intel Corporation. **Intel® Software Guard Extensions Attestation Services.** Official documentation on SGX remote attestation mechanisms including ECDSA-based attestation and DCAP architecture. 2023. URL: <https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/attestation-services.html> (visited on 06/22/2025).

- [Int25] Intel Corporation. **Intel Trust Domain Extensions (TDX)**. White paper. Overview of TD architecture, SEAM, memory/CPU-state protection, attestation. Intel, June 2025. URL: <https://cdrdv2-public.intel.com/856790/Intel%20Trust%20Domain%20Extensions%20Overview%20-%20June%202025.pdf>.
- [Kol22] Kevin Kollenda. **General overview of amd sev-snp and intel tdx. Friedrich-Alexander-Universität Erlangen-Nürnberg, Technical Report** (2022).
- [Kum+19] Nishant Kumar et al. **CrypTFlow: Secure TensorFlow Inference. IACR Cryptol. ePrint Arch.** (2019), p. 1049. URL: <https://eprint.iacr.org/2019/1049>.
- [Lut+25] Adrian Lutsch et al. **An Analysis of AWS Nitro Enclaves for Database Workloads.** Proceedings of the 21st International Workshop on Data Management on New Hardware, DaMoN 2025, Berlin, Germany, June 22-27, 2025. ACM, 2025, 5:1–5:8. DOI: 10.1145/3736227.3736234. URL: <https://doi.org/10.1145/3736227.3736234>.
- [Mis+24] Masanori Misono et al. **Confidential vms explained: An empirical analysis of amd sev-snp and intel tdx. Proceedings of the ACM on Measurement and Analysis of Computing Systems** 8.3 (2024), pp. 1–42.

- [TAC24a] TACEO. **Best Practices - TACEO:Proof**. Accessed: 2026-04-04. 2024. URL: <https://docs.taceo.io/docs/taceo-proof/dev/bestpractice/>.
- [TAC24b] TACEO GmbH. **co-snarks: Tooling for Collaborative SNARKs**. <https://github.com/TaceoLabs/co-snarks>. Accessed: 2025-07-30. 2024.
- [Urb24] Antoine Urban. **Efficient delegated secure multiparty computation. (Délégation efficace de calcul multipartite sécurisé)**. PhD thesis. Polytechnic Institute of Paris, Palaiseau, France, 2024. URL: <https://tel.archives-ouvertes.fr/tel-04955820>.

Secure Multi-Party Computation with Trusted Execution Environments

Sebastian David Albu

June 25, 2026